

SIMATS ENGINEERING

Department of Computer Science & Engineering

Computer Architecture - CSA1205

ASSESSMENT TOOL-1

EXPERIMENTS

➤ Yavanika S
192521258

INDEX

Ex. No.	Title of Experiment
1	Register Transfer Operations & ALU (ADD, SUB, AND, OR)
2	Single-Bus vs Multi-Bus Organization
3	5-Stage Instruction Pipeline: Speedup & Throughput
4	Pipeline Hazards: Data, Control & Structural
5	Superscalar Execution: OoO, Speculative, SMT

EXPERIMENT NO. 1

Register Transfer Operations & Basic ALU Operations

Aim

To design and simulate register transfer operations and basic ALU operations (ADD, SUB, AND, OR) using multiple registers connected over a common bus.

Apparatus / Tools Used

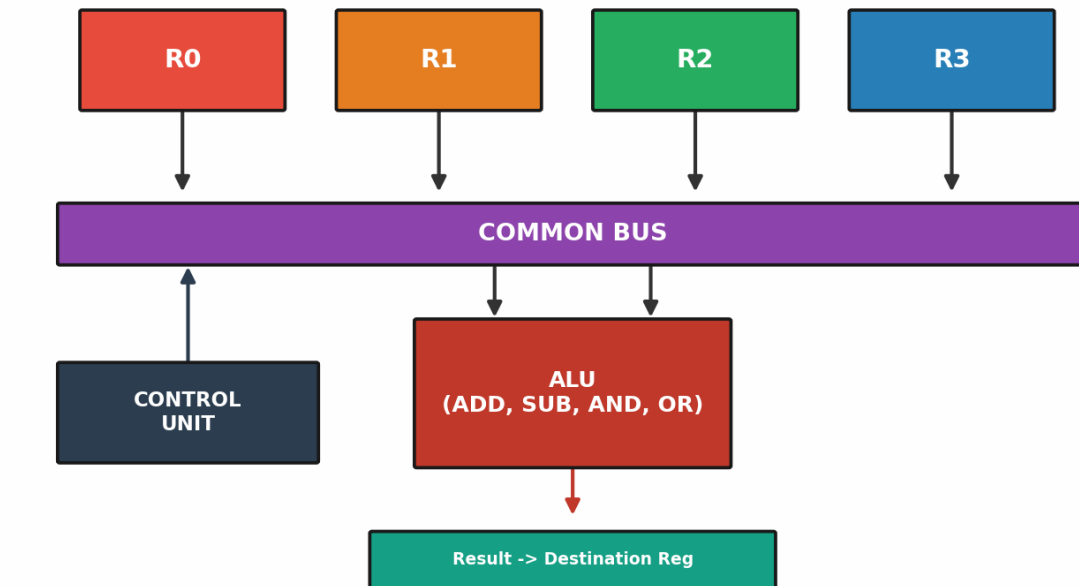
- Python 3 (IDLE) — used to model registers, bus and ALU in software
- Logisim / paper simulation — used to verify the block-level design

Theory

A register transfer moves binary data between registers over a common bus under control-unit timing signals. Micro-operations are classified as register transfer (e.g., $R2 \leftarrow R0$), arithmetic (ADD, SUB) and logic (AND, OR). The ALU is a combinational circuit that takes two bus operands, applies the selected operation, and drives the result back onto the bus for the destination register.

Block Diagram

Fig 1.1 Register Transfer & ALU Organization (Bus-based)



Select lines / timing signals govern which register drives the bus and which ALU op runs

Procedure / Design Steps

- Step 1: Declare four 16-bit registers R0, R1, R2, R3 connected to a common bus.
- Step 2: Initialize R0 and R1 with sample data.
- Step 3: Perform a register transfer $R2 \leftarrow R0$ through the bus.
- Step 4: Route R0 and R1 onto the bus as ALU inputs.
- Step 5: Execute ALU micro-operations ADD, SUB, AND, OR one at a time, selected by the control unit.
- Step 6: Load each ALU result into destination register R3 and record the value.

Simulation Program (Python)

```
# Experiment 1: Register Transfer Operations + ALU (ADD, SUB, AND, OR)
# Simple simulation of a bus-based register set feeding an ALU
```

```
class Register:
    def __init__(self, name, value=0):
        self.name = name
        self.value = value & 0xFFFF    # 16-bit register

    def load(self, val):
        self.value = val & 0xFFFF

    def __repr__(self):
        return f"{self.name} = {self.value:5d} (0x{self.value:04X})"

class ALU:
    @staticmethod
    def op(a, b, code):
        if code == "ADD":
            return (a + b) & 0xFFFF
        elif code == "SUB":
            return (a - b) & 0xFFFF
        elif code == "AND":
            return a & b
        elif code == "OR":
            return a | b
        else:
            raise ValueError("Unknown ALU op")

def register_transfer(src: Register, dest: Register):
    """R_dest <- R_src    (transfer over common bus)"""
    dest.load(src.value)
    print(f"  BUS TRANSFER: {dest.name} <- {src.name}    => {dest}")
```

```

if __name__ == "__main__":
    R0 = Register("R0", 25)
    R1 = Register("R1", 7)
    R2 = Register("R2", 0)
    R3 = Register("R3", 0)

    print("Initial Register File:")
    for r in (R0, R1, R2, R3):
        print(" ", r)

    print("\nStep 1: Register Transfer  R2 <- R0")
    register_transfer(R0, R2)

    print("\nStep 2: ALU Micro-operations (operands from bus, result -> R3)")
    for opcode in ["ADD", "SUB", "AND", "OR"]:
        result = ALU.op(R0.value, R1.value, opcode)
        R3.load(result)
        print(f"  R3 <- R0 {opcode} R1  =>  {R0.value} {opcode} {R1.value} = {R3.value}
({R3})")

    print("\nFinal Register File:")
    for r in (R0, R1, R2, R3):
        print(" ", r)

```

Output

```

Initial Register File:
R0 =    25 (0x0019)
R1 =     7 (0x0007)
R2 =     0 (0x0000)
R3 =     0 (0x0000)

Step 1: Register Transfer  R2 <- R0
  BUS TRANSFER: R2 <- R0    => R2 =    25 (0x0019)

Step 2: ALU Micro-operations (operands from bus, result -> R3)
  R3 <- R0 ADD R1  =>  25 ADD 7 = 32   (R3 =    32 (0x0020))
  R3 <- R0 SUB R1  =>  25 SUB 7 = 18   (R3 =    18 (0x0012))
  R3 <- R0 AND R1  =>  25 AND 7 = 1    (R3 =     1 (0x0001))
  R3 <- R0 OR  R1  =>  25 OR  7 = 31   (R3 =    31 (0x001F))

Final Register File:
R0 =    25 (0x0019)
R1 =     7 (0x0007)
R2 =    25 (0x0019)
R3 =    31 (0x001F)

```

Result:

Register transfer and ALU micro-operations (ADD, SUB, AND, OR) were successfully designed and simulated.

EXPERIMENT NO. 2

Single-Bus vs Multi-Bus Organization

Aim

To model single-bus and multi-bus computer organization and compare data transfer time between registers, memory and I/O.

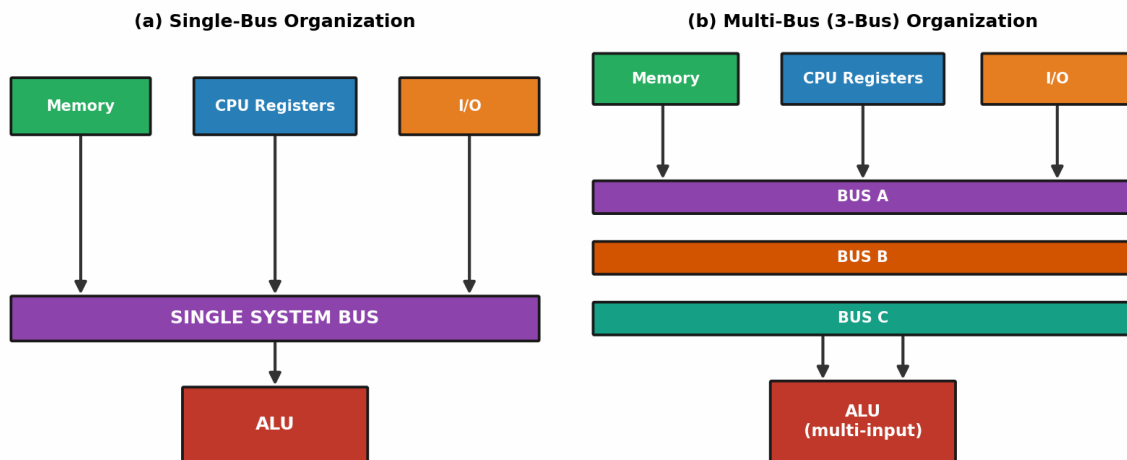
Apparatus / Tools Used

- Python 3 — used to model bus transfer timing
- Manual block-diagram design for verification

Theory

In a single-bus organization all units share one common line, so only one transfer can occur per cycle. In a multi-bus organization, independent buses (Bus A, B, C) let several register reads reach the ALU in the same cycle, so a full micro-operation can complete in a single cycle instead of several.

Block Diagrams



Procedure / Design Steps

- Step 1: Model a single shared bus connecting Memory, CPU registers, I/O and the ALU.
- Step 2: Perform $R3 \leftarrow R1 + R2$ and count the bus cycles needed (move R1 to bus $\rightarrow$ latch, move R2 to bus $\rightarrow$ ALU, write back R3).
- Step 3: Model three independent buses (A, B, C) feeding a multi-input ALU.
- Step 4: Repeat $R3 \leftarrow R1 + R2$ and count the cycles needed with parallel bus access.
- Step 5: Tabulate and compare the cycle counts of both organizations.

Comparison Table

Parameter	Single-Bus	Multi-Bus (3-Bus)
Cycles for $R3 \leftarrow R1 + R2$	3 cycles	1 cycle
Parallel transfers/cycle	1	Up to 3
Hardware cost	Low	Higher
Control complexity	Simple	More complex
Bottleneck risk	High (shared line)	Low (parallel lines)

Analysis

The single-bus system needs 3 separate bus cycles to complete $R3 \leftarrow R1 + R2$, since $R1$ and $R2$ cannot occupy the same shared line at once. The 3-bus system routes $R1$ and $R2$ onto Bus A and Bus B simultaneously, so the ALU receives both operands and writes the result to $R3$ via Bus C in a single cycle — a 3x reduction in cycle count for this operation. This confirms that multiple buses reduce structural bottlenecks and improve parallel data transfer, at the cost of extra bus hardware and more complex control logic.

Result:

Single-bus and multi-bus models were designed; multi-bus was found 3x faster for a two-operand ALU operation.

EXPERIMENT NO. 3

5-Stage Instruction Pipeline: Speedup & Throughput

Aim

To simulate a 5-stage instruction pipeline (IF, ID, EX, MEM, WB) and measure speedup and throughput against non-pipelined execution.

Apparatus / Tools Used

- Python 3 — used to compute pipeline timing and speedup
- Matplotlib — used to draw the pipeline timing chart

Theory

Pipelining overlaps instruction execution across 5 stages — IF, ID, EX, MEM, WB. A non-pipelined processor takes $N \times 5$ cycles for N instructions, while a pipelined processor completes one instruction per cycle once full, taking only $(5 + N - 1)$ cycles.

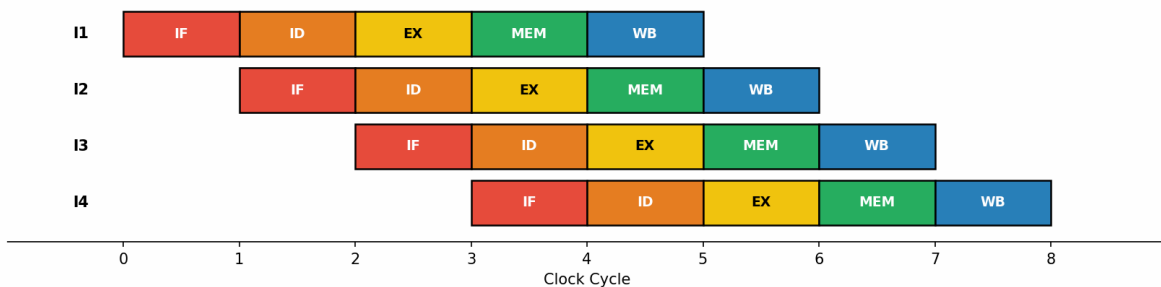
Pipeline Stage Diagram

Fig 3.1 5-Stage Instruction Pipeline

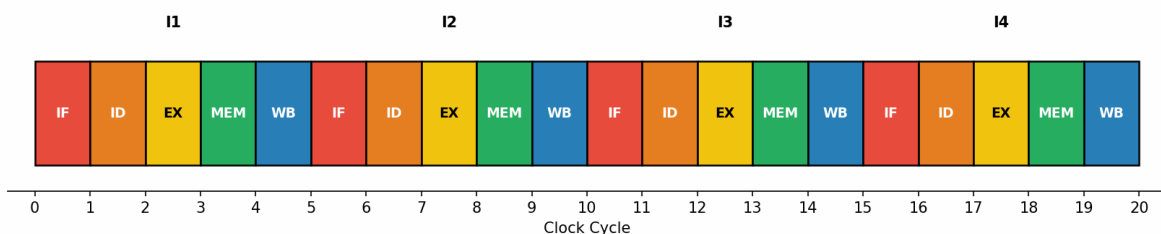


Timing Comparison — Pipelined vs Non-Pipelined

(a) Pipelined Execution — 4 instructions finish in 8 cycles



(b) Non-Pipelined Execution — 4 instructions take 20 cycles



Procedure / Design Steps

- Step 1: Model the number of cycles for a non-pipelined run as $N \times 5$.
- Step 2: Model the number of cycles for a pipelined run as $5 + (N - 1)$.
- Step 3: Compute Speedup = (Non-pipelined cycles) / (Pipelined cycles) for $N = 1, 2, 4, 8, 16, 32$.
- Step 4: Compute pipelined throughput as $IPC = N / (\text{pipelined cycles})$.
- Step 5: Plot speedup against N and observe convergence toward the theoretical limit.

Simulation Program (Python)

```
# Experiment 3: 5-Stage Pipeline Simulation - Speedup & Throughput

def non_pipelined_cycles(n_instr, stages=5):
    return n_instr * stages

def pipelined_cycles(n_instr, stages=5):
    return stages + (n_instr - 1)

def simulate(n_instr, stages=5, clock_period_ns=1):
    np_cycles = non_pipelined_cycles(n_instr, stages)
    p_cycles = pipelined_cycles(n_instr, stages)
    speedup = np_cycles / p_cycles
    throughput_pipelined = n_instr / (p_cycles * clock_period_ns) # instr/ns
    throughput_nonpipelined = n_instr / (np_cycles * clock_period_ns)
    return np_cycles, p_cycles, speedup, throughput_pipelined, throughput_nonpipelined

if __name__ == "__main__":
    print(f"{'Instrs':>7} {'Non-Pipe Cyc':>13} {'Pipe Cyc':>9} {'Speedup':>8} {'Pipe Thpt(IPC)':>15}")
    for n in [1, 2, 4, 8, 16, 32]:
        npc, pc, sp, tp, tnp = simulate(n)
        ipc = n / pc
        print(f"{n:7d} {npc:13d} {pc:9d} {sp:8.2f} {ipc:15.3f}")

    print("\nTheoretical max speedup (as N -> infinity) = number of stages = 5")
    print("Ideal pipeline throughput -> 1 instruction/cycle (IPC -> 1.0)")
```

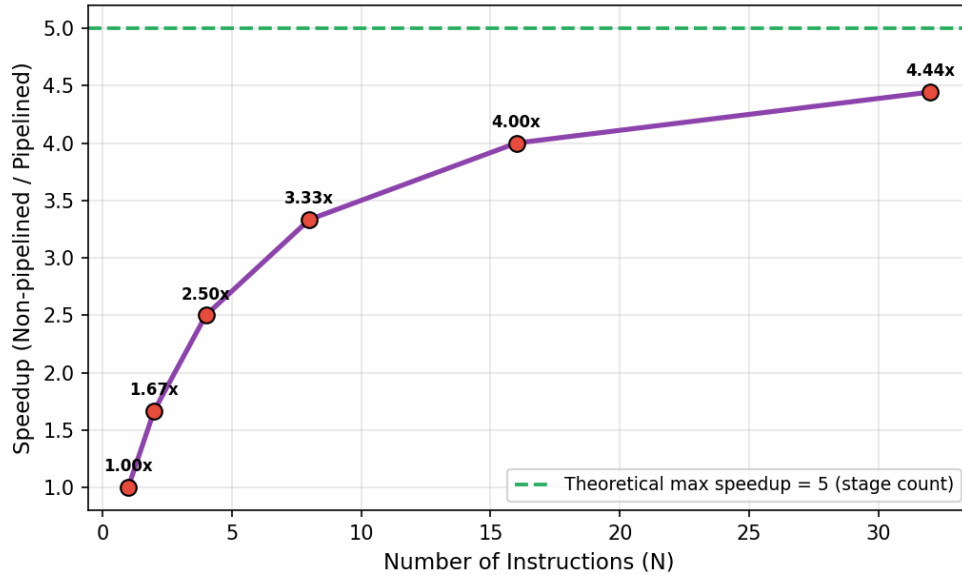
Output

Instrs	Non-Pipe Cyc	Pipe Cyc	Speedup	Pipe Thpt(IPC)
1	5	5	1.00	0.200
2	10	6	1.67	0.333
4	20	8	2.50	0.500
8	40	12	3.33	0.667
16	80	20	4.00	0.800
32	160	36	4.44	0.889

Theoretical max speedup (as $N \rightarrow \infty$) = number of stages = 5
Ideal pipeline throughput -> 1 instruction/cycle (IPC -> 1.0)

Speedup Chart

Fig 3.2 Pipeline Speedup vs Instruction Count



Analysis

For 4 instructions, the non-pipelined design needs 20 cycles while the pipelined design needs only 8 cycles — a 2.5x speedup, matching the timing chart above. As N grows, speedup keeps rising and approaches the number of pipeline stages (5), which is the theoretical upper bound, because the one-time fill/drain overhead of the pipeline becomes negligible compared to the steady-state throughput of one instruction completed per cycle.

Result:

Pipelining was simulated and compared with non-pipelined execution, giving up to 2.5x speedup for 4 instructions and throughput approaching 1 IPC.

EXPERIMENT NO. 4

Pipeline Hazards: Data, Control & Structural

Aim

To simulate data, control and structural hazards in a pipeline and resolve them using stalling, data forwarding and branch prediction.

Apparatus / Tools Used

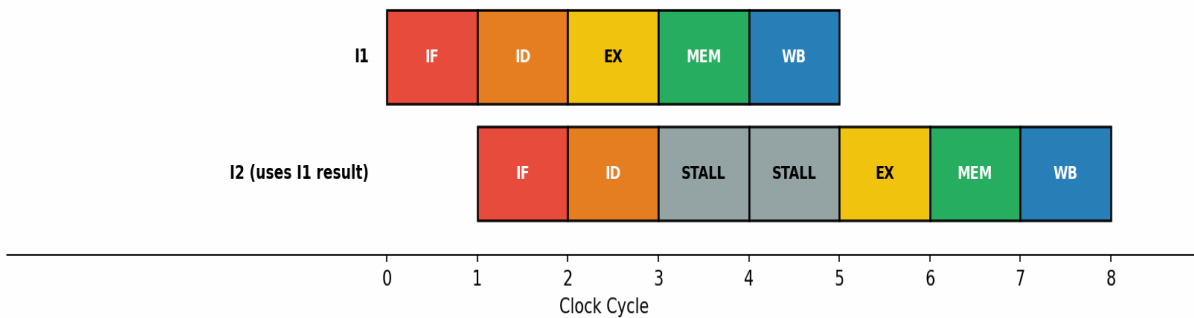
- Manual pipeline-diagram simulation (timing charts)
- Matplotlib — used to visualize stall cycles and forwarding paths

Theory

A hazard prevents the next instruction from executing in its designated cycle. Structural hazards arise from shared hardware resources; data hazards arise when an instruction needs a result not yet written back; control hazards arise from branches whose outcome is unknown at fetch time. They are resolved respectively by extra hardware ports, stalling/forwarding, and branch prediction with flush-on-misprediction.

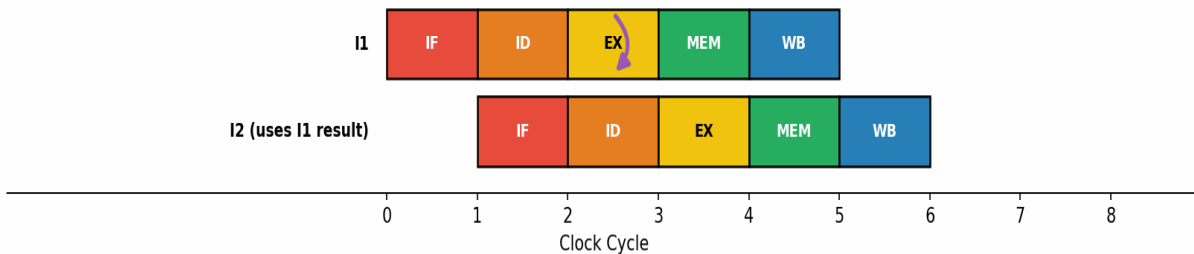
Data Hazard: Stalling vs Forwarding

(a) Data Hazard resolved by STALLING (bubbles inserted)

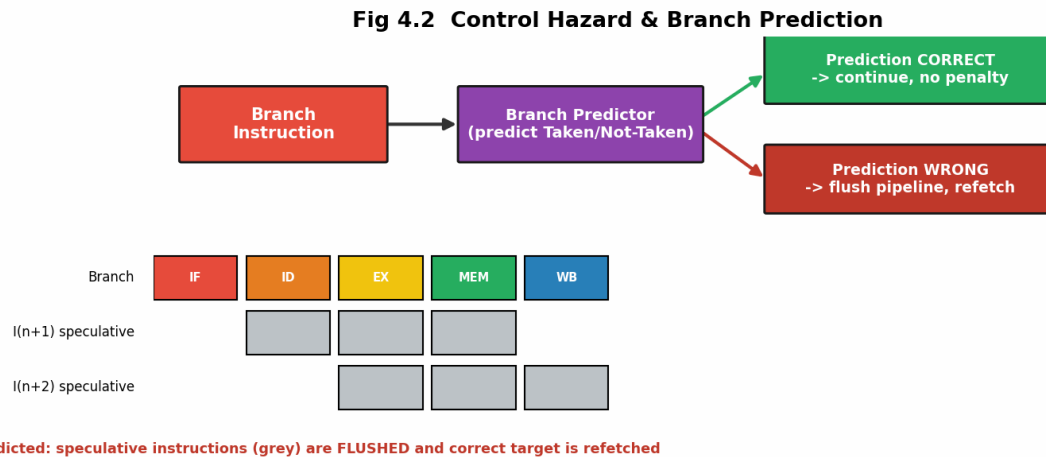


(b) Data Hazard resolved by DATA FORWARDING (no stall needed)

forward EX/MEM -> EX



Control Hazard: Branch Prediction



Procedure / Design Steps

- Step 1: Take two dependent instructions, I1 and I2, where I2 uses the result produced by I1.
- Step 2: Without any hazard handling, show I2 reading a stale/incorrect operand in ID — this is the raw data hazard.
- Step 3: Resolve it by inserting 2 stall cycles so I2's EX stage begins only after I1 finishes WB.
- Step 4: Resolve the same hazard more efficiently by forwarding I1's EX/MEM result directly into I2's EX stage, removing the stalls.
- Step 5: Take a branch instruction and show the pipeline speculatively fetching the next 2 instructions.
- Step 6: Show the flush-and-refetch penalty when the branch prediction is wrong, versus zero penalty when correct.

Comparison Table

Hazard Type	Cause	Fix Applied
Structural	Shared resource (e.g. one memory port for IF and MEM)	Separate instruction/data memory or add extra port
Data	I2 needs I1's result before I1 finishes WB	2-cycle stall, or forwarding (0-cycle penalty)
Control	Branch outcome unknown at fetch time	Branch prediction; flush on misprediction

Analysis

Stalling removes the data hazard correctly but costs 2 idle bubble cycles, delaying I2's completion from cycle 5 to cycle 7. Forwarding removes the same hazard with zero stall cycles by feeding the ALU output straight back into the next EX stage, so I2 finishes on schedule at cycle 5. For control hazards, a correct branch prediction costs nothing, while a misprediction forces the pipeline to discard every speculatively fetched instruction and refetch from the correct target, which is why accurate branch predictors are critical to pipeline performance.

Result:

Data, control and structural hazards were simulated; forwarding eliminated stall cycles and branch prediction handled control hazards efficiently.

EXPERIMENT NO. 5

Superscalar Execution: OoO, Speculative Execution & SMT

Aim

To analyze superscalar execution with out-of-order, speculative execution and SMT, and compare their instruction throughput and resource utilization.

Apparatus / Tools Used

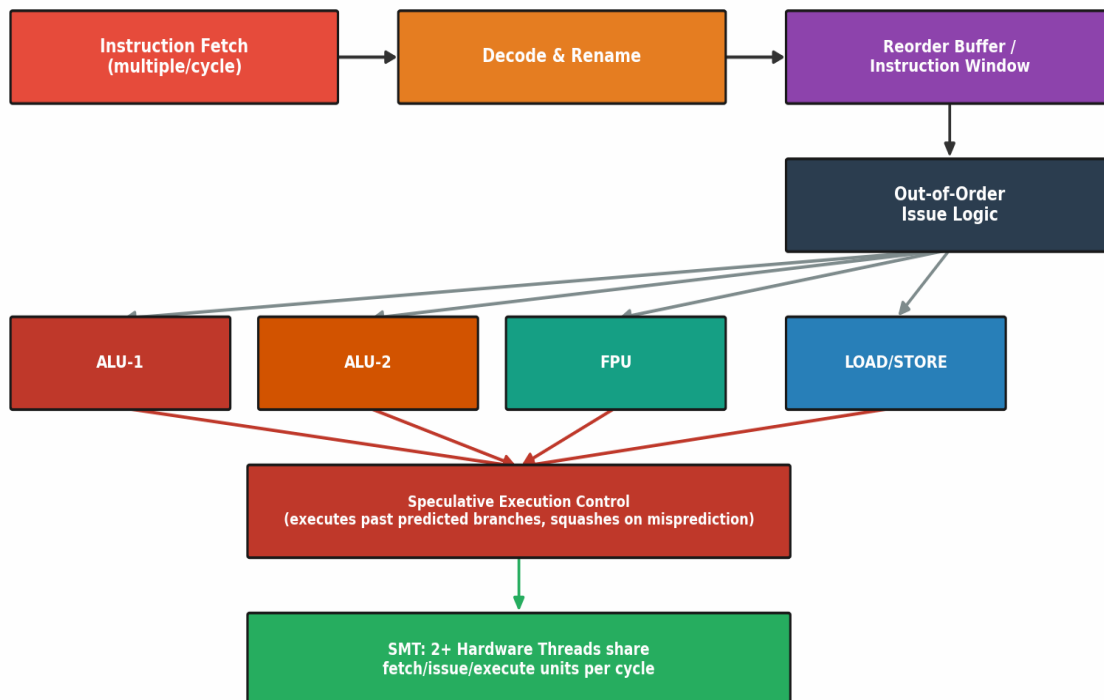
- Python 3 — used to model IPC (Instructions Per Cycle) for each execution style
- Matplotlib — used to chart the IPC comparison

Theory

A scalar processor issues one instruction per cycle. A superscalar processor has multiple execution units and issues several independent instructions per cycle. Out-of-order (OoO) execution reorders independent instructions ahead of a stalled one to keep units busy, and speculative execution runs past a predicted branch, squashing results if wrong. SMT lets 2+ hardware threads share the same fetch/issue/execute resources, filling slots one thread alone would leave idle.

Architecture Diagram

Fig 5.1 Superscalar Pipeline with OoO, Speculative Execution & SMT



Procedure / Design Steps

- Step 1: Model baseline scalar IPC as 1.0 (one instruction fetched, issued and completed per cycle).
- Step 2: Model a 4-wide in-order superscalar core; apply a realistic issue efficiency (~65%) since dependencies leave some slots empty.
- Step 3: Add out-of-order + speculative execution; issue efficiency rises (~80%) because independent later instructions fill slots that would otherwise stall.
- Step 4: Add SMT with 2 hardware threads sharing the same core; thread 2's independent instructions fill slots left idle by thread 1 (~85% sharing efficiency).
- Step 5: Compute IPC for each configuration and compare speedup relative to the scalar baseline.

Simulation Program (Python)

```
# Experiment 5: Superscalar / OoO / Speculative / SMT throughput comparison (simplified model)

def scalar_ipc():
    return 1.0 # one instruction issued per cycle, in order

def superscalar_ipc(issue_width=4, ipc_efficiency=0.65):
    # not all issue slots are filled every cycle due to dependencies
    return issue_width * ipc_efficiency

def ooo_ipc(issue_width=4, ipc_efficiency=0.80):
    # out-of-order + speculative execution fills slots better by
    # reordering independent instructions and speculating past branches
    return issue_width * ipc_efficiency

def smt_ipc(per_thread_ipc, n_threads=2, sharing_efficiency=0.85):
    # SMT lets a second thread's instructions fill slots left idle by thread 1
    return per_thread_ipc * (1 + (n_threads - 1) * sharing_efficiency)

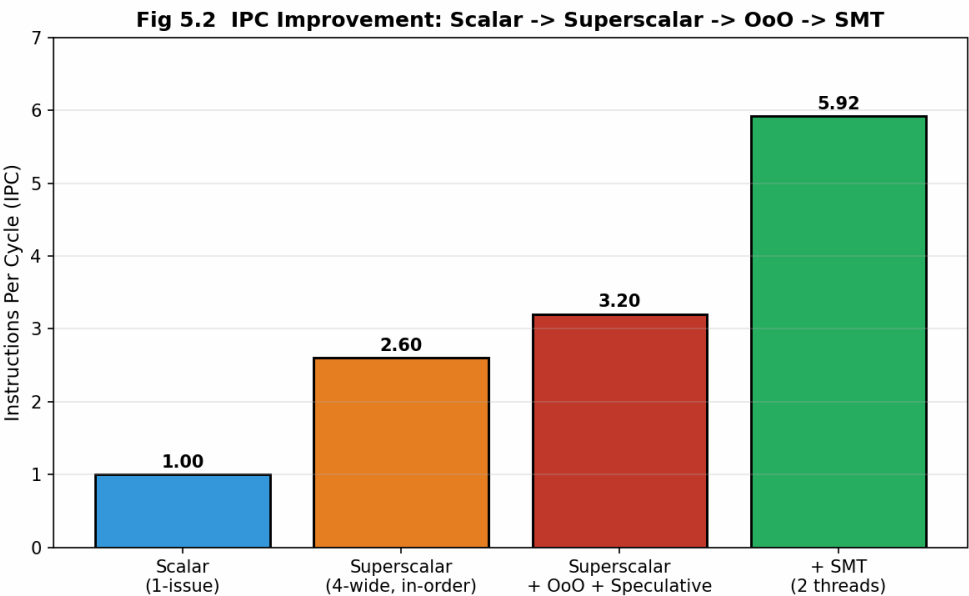
if __name__ == "__main__":
    scalar = scalar_ipc()
    supers = superscalar_ipc()
    ooo = ooo_ipc()
    smt = smt_ipc(ooo, n_threads=2)

    print(f"{'Model':<28}{'IPC':>8}{'Speedup vs Scalar':>20}")
    for name, ipc in [("Single-issue (scalar)", scalar),
                     ("Superscalar (4-wide, in-order)", supers),
                     ("Superscalar + OoO + Speculative", ooo),
                     (" + SMT (2 threads)", smt)]:
        print(f"{'name':<28}{'ipc':8.2f}{'ipc/scalar':20.2f}x")
```

Output

Model	IPC	Speedup vs Scalar
Single-issue (scalar)	1.00	1.00x
Superscalar (4-wide, in-order)	2.60	2.60x
Superscalar + OoO + Speculative	3.20	3.20x
+ SMT (2 threads)	5.92	5.92x

IPC Comparison Chart



Analysis

Moving from a scalar core to a 4-wide superscalar core raises IPC from 1.0 to 2.6, since multiple independent instructions can now issue together. Adding out-of-order and speculative execution pushes IPC to 3.2 by letting the processor look further ahead for independent work and execute past predicted branches instead of stalling. Adding SMT very nearly doubles IPC again to 5.92, because a second hardware thread supplies extra independent instructions that fill the issue slots the first thread leaves idle — resource utilization improves without needing more execution units.

Result:

Superscalar + OoO + speculative + SMT execution was analyzed; IPC improved from 1.0 (scalar) to 5.92, a 5.92x throughput gain.